



类和对象

北京理工大学计算机学院
金旭亮

1 类的设计



编写类的“模板”



```
[public] class 类名 {  
    [public|private] 数据类型 变量名;  
    [public|private] 数据类型 方法名(参数列表){  
    }  
}
```

两种最基本的类成员存取权限:

1. **public:** 存取与访问不受限制
2. **private:** 除非是属于类自己的方法，外界代码不能存取和访问

定义Java类的示例

某些编程语言（比如C#）为“**属性（property）**”提供了单独的定义方式，在Java中，我们可以通过组合一个私有字段和一对get/set方法来定义一个属性。

```
/**
 * 自定义Java类的示例
 */
public class MyClass {
    //公有字段
    public String information = "";

    //自定义公有Java实例方法
    public void myMethod(String argu) {
        System.out.println(argu);
    }

    //region "属性 = 私有字段 + get方法 + set方法"
    private int value;

    public int getValue() {
        return value;
    }

    public void setValue(int value) {
        this.value = value;
    }
    //endregion
}
```

使用自定义类

```
class UseMyClass{  
    public static void main(String[] args) {  
        //创建类的实例  
        MyClass obj=new MyClass();  
        //通过对象变量调用类的公有方法  
        obj.myMethod( argu: "Hello");  
        //给属性赋值  
        obj.setValue(100);  
        //输出属性的当前值  
        System.out.println(obj.getValue());  
        //直接访问对象公有字段  
        obj.information="Information";  
        //输出对象公有字段的当前值  
        System.out.println(obj.information);  
    }  
}
```

示例：MyClass.java

小结



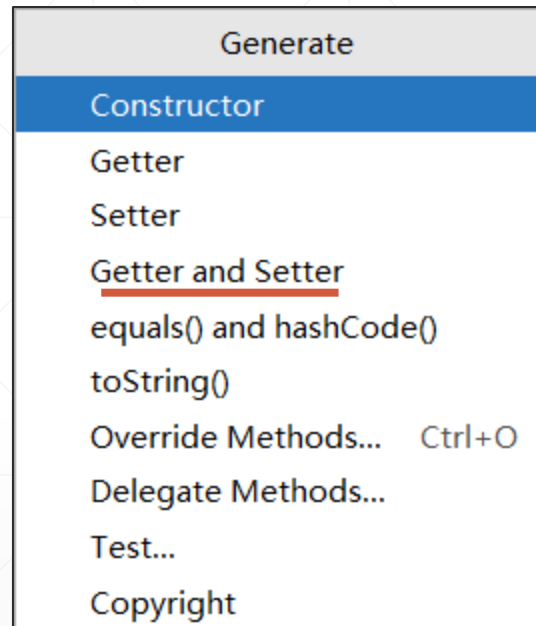
从上一页示例中，我们可以总结出以下知识点：

1. 我们需要定义一个对象变量
2. 然后“**创建 (new)**”一个对象，赋值给这个对象变量
3. 现在就可以通过这个对象变量使用对象，主要方式有：
 - ① 直接调用类定义的**公有 (public)** 方法
 - ② 存取类定义的**公有** 字段

小技巧



在IntelliJ中编写类的代码时，使用 **Alt + Ins** 组合键，会弹出一个快捷菜单，其中包容许多常用的命令，可以方便地生成各种框架代码，能够极大地提升编码效率。





2 对象的内存模型

一种新的变量类型

早期我们经常这样定义变量：

```
int value = 100;
```

前面的示例中这样定义变量：

```
MyClass obj = new MyClass();
```



这两种方式定义的变量有什么区别吗？



引用类型 vs. 原始数据类型



“引用”一个对象的变量称为“**引用类型**”的变量，有时又简称为“**对象变量**”。



诸如 int, float 之类的变量则称为“**原始数据类型**”的变量。

变量与内存分配



当声明一个对象类型的变量时,实际上并没有创建一个对象,此变量值为默认值 `null`。



定义一个原始数据类型的变量时,则会马上给其分配内存并初始化(如果没有指定,则为默认值,通常是0)

`MyClass obj = null ;`

`obj` ↔

`null`

`int value = 100 ;`

`value` ↔

`100`

变量在使用前必须初始化

在定义变量时，Java要求必须显式初始化变量，比如以下代码将无法通过编译：

```
public class Test {  
  
    public static void main(String[] args) {  
        int value;  
        System.out.println(value);  
    }  
}
```



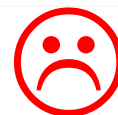
```
public class Test {  
  
    public static void main(String[] args) {  
        int value=100;  
        System.out.println(value);  
    }  
}
```



对象变量的初始化

对象变量如果不引用一个真实的对象，则必须声明为null:

```
public class Test {  
  
    public static void main(String[] args) {  
        MyClass obj;  
        System.out.println(obj.toString());  
    }  
}
```



```
public class Test {  
  
    public static void main(String[] args) {  
        MyClass obj=null;  
        System.out.println(obj.toString());  
    }  
}
```

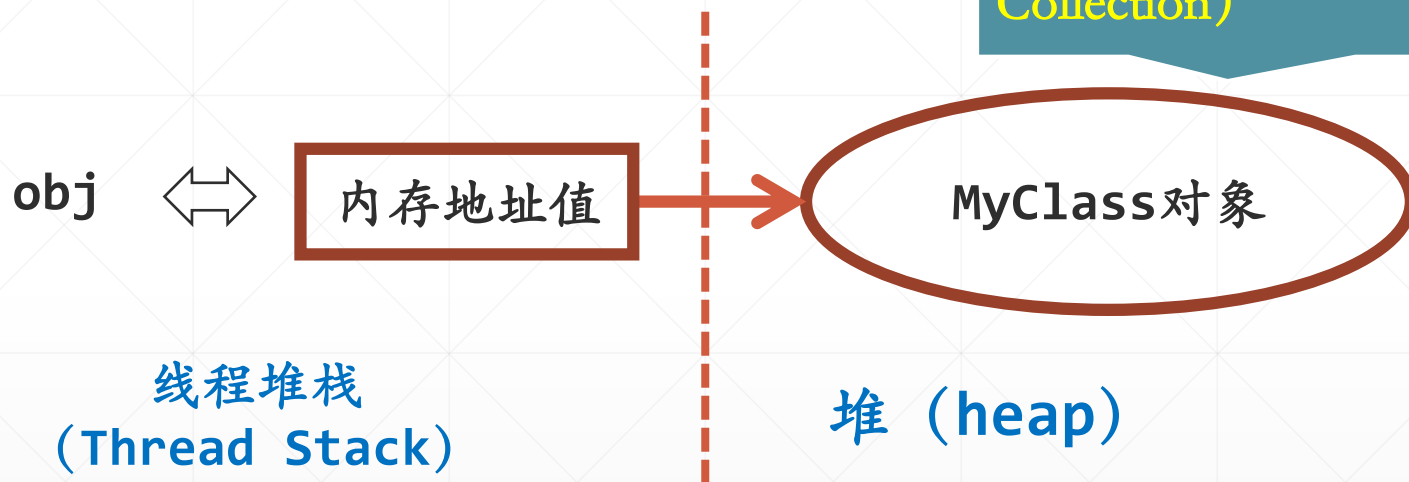


对象变量的特殊值null: 代表一个对象变量不引用任何对象。

引用对象后的对象变量

```
MyClass obj = new MyClass();
```

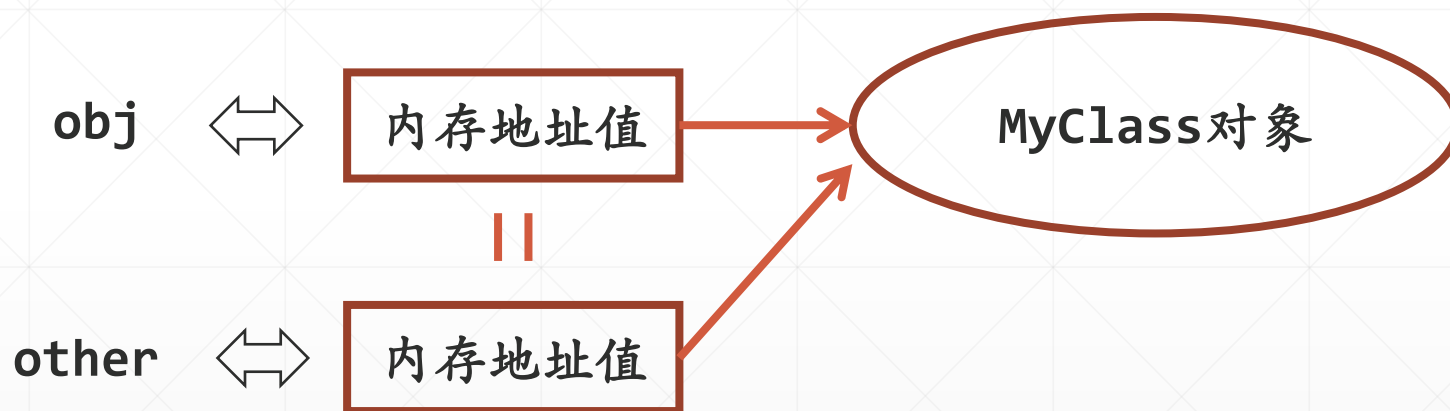
当对象不再使用时，JVM会回收
MyClass对象所占用的内存，这个过程
称为“**垃圾回收 (GC: Garbage
Collection)**”



`new`这一关键字的作用，就是通知JVM为创建出来的新对象分配足够量的内存（在堆中），并自动调用此对象的一个特定方法（称为“**构造方法**”）对这块内存进行初始化。

理解赋值“=”的含义

```
MyClass obj = new MyClass();  
MyClass other = obj;
```

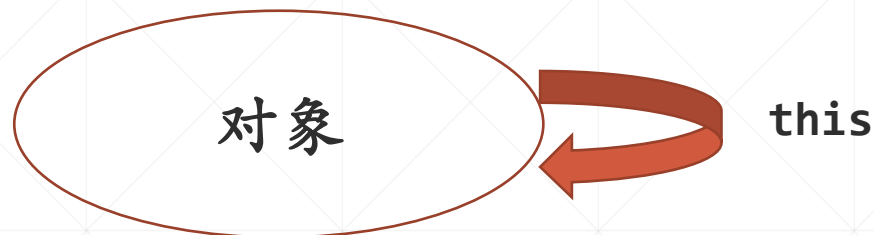


对象变量之间的赋值，本质上是指向对象的那个内存地址值的复制（而不是对象本身的复制）。赋值结束之后，两个对象变量引用同一个对象。

独特的对象引用: this



对象中的所有数据字段可以通过this间接引用。



同一类中的方法可以相互调用，或者直接存取本类定义的字
段，可看成其中隐含了一个this调用。

```
class MyClass {  
    int value = 0;  
  
    void f() {  
    }  
  
    void g() {  
        value = 100;  
        f();  
    }  
}
```



```
class MyClass {  
    int value = 0;  
  
    void f() {  
    }  
  
    void g() {  
        this.value = 100;  
        this.f();  
    }  
}
```

练习

```
public class ClassFieldAndLocalVariable {  
    public static void main(String[] args) {  
        var obj = new VariableAndField( value: 10);  
        obj.doSomething();  
    }  
}
```

```
class VariableAndField {  
    private int x;  
  
    public VariableAndField(int value) {  
        this.x = value;  
    }  
  
    public void doSomething() {  
        for (int x = 0; x < 5; x++) {  
            System.out.println(x + " * " + x + " = " + x * x);  
            //System.out.println(x + " * " + this.x + " = " + x * this.x);  
        }  
    }  
}
```

请对比这两句代码的输出有什么不同？

ClassFieldAndLocalVariable.java

作为常量的对象变量

可以使用`final`定义一个“常量”对象变量：

```
final MyClass obj = new MyClass();
```



注意：“常量”对象变量不能引用另一个对象，但可以修改对象所包容的数据，比如修改它所引用的那个对象的某个公有字段值为另一个值。

判等操作



对于原始数据类型的变量（比如int），可以直接使用“==”运算符判断两变量值是否相等：

```
public class Test {  
  
    public static void main(String[] args) {  
        int value1=100;  
        int value2=100;  
        System.out.println(value1==value2);//true  
    }  
}
```



对象变量也可以使用“==”判断两变量所引用的对象，其值是否相等吗？

进行试验



请输入并运行以下代码，看看得到什么结果.....

```
public class Test {  
    public static void main(String[] args) {  
        Foo obj1=new Foo();  
        Foo obj2=new Foo();  
        System.out.println(obj1==obj2);//?  
    }  
}  
  
class Foo{  
    int value=100;  
}
```

实践出真知



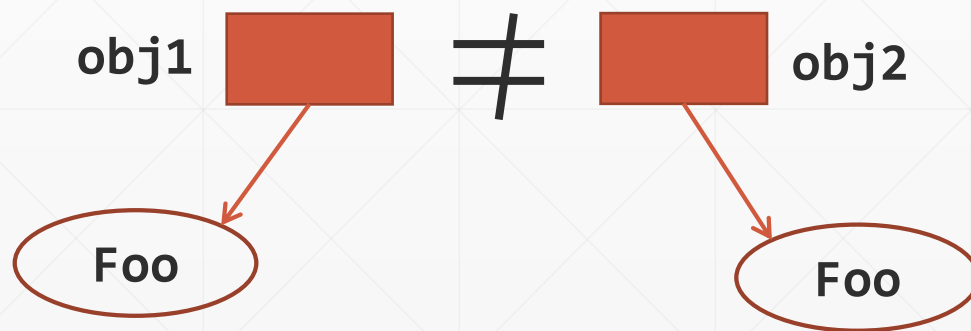
当“==”施加于原始数据类型的变量时，是比较变量所保存的数据“**值**”是否相等。



当“==”施加于引用类型变量时，是比较这两个变量是否**引用**同一对象。



引用代表地址，所以“==”实际上相当于比较两个引用类型变量中保存的对象地址数值是否相同。



如何比较两个对象的“内容”是否一样？



两个对象的“内容”，其实是指它们在某一时刻的所有字段的值，“内容相等”，也就是两个对象的“对应字段值”一致。



在Java中要比对两个对象的字段值，需要“**重写 (override)**”基类的**equals()**方法

示例：ObjectEquals.java

重写基类的equals方法

```
class MyTestClass {  
    public int Value;  
    @Override  
    public boolean equals(Object obj) {  
        return ((MyTestClass) obj).Value == this.Value;  
    }  
    public MyTestClass(int initValue) {  
        Value = initValue;  
    }  
}
```

左边这种重写equals()的方式是简化的，在实际开发中不应该这样写，至少应该加上对参数有效性的检测代码，另外还需要重写另一个获取对象hash值的方法。

测试运行结果的代码

```
public class ObjectEquals {  
  
    public static void main(String[] args) {  
        MyTestClass obj1 = new MyTestClass(100);  
        MyTestClass obj2 = new MyTestClass(100);  
        System.out.println(obj1 == obj2); //false  
        System.out.println(obj1.equals(obj2)); //true  
    }  
}
```

"@Override"指明此方法是"重写基类的同名方法", Java中, 这种以"@ "打头的标记被称为“**注解 (Annotation)**”。

equals方法从哪来?



当定义一个类时，如果不显式指明它的“父类”是哪个，则默认是Object。



Object是Java的**最顶层基类**，其中定义了equals()方法

```
Object
  Object()
  registerNatives() : void
  getClass() : Class<?>
  hashCode() : int
  equals(Object) : boolean
  clone() : Object
  toString() : String
  notify() : void
  notifyAll() : void
  wait(long) : void
  wait(long, int) : void
  wait() : void
  finalize() : void
```



Object类的成员

自定义equals()方法



除了重写基类的equals方法，也可以“**重载 (overload)**”
equals()方法

```
class MyTestClass {  
    public int Value;  
  
    public boolean equals(MyTestClass obj) {  
        return obj.Value == this.Value;  
    }  
  
    public MyTestClass(int initValue) {  
        Value = initValue;  
    }  
}
```

注意：
与前面“重写”方式的代码相比，
equals()方法的参数类型是
MyTestClass而不是Object，并且方
法本身也没有附加“@Override”
注解。

Objects.equals()方法-1

JDK 7中引入了一个Objects工具类，其中定义了一个equals()方法，其源码如下：

```
public static boolean equals(Object a, Object b) {  
    return (a == b) || (a != null && a.equals(b));  
}
```

从其源码可知，类是否重写了equals方法，会直接影响到其比对的结果。

Objects.equals()方法-2

```
class MyTestClass {  
    public int Value;  
  
    // 注意：只有参数类型为Object的，才是重写了Object的equals方法  
    // 参数类型为MyTestClass的，仅仅是Overload了equals方法。  
    // 这个方法是否被注释，会影响到Objects.equals的结果  
    // @Override  
    // public boolean equals(Object obj) {  
    //     return ((MyTestClass) obj).Value == this.Value;  
    // }  
  
    public boolean equals(MyTestClass obj) {  
        return obj.Value == this.Value;  
    }  
  
    public MyTestClass(int initValue) { Value = initValue; }  
}
```

ObjectEquals.java

左侧的类中，是否取消掉注释的代码，会导致Objects.equals方法返回不同的结果，请打开示例文件运行一下。

小结



本讲包容丰富的内容，介绍的知识也非常关键，它们是顺利学会学懂后继内容的前提。



请同学们务必理解清楚本讲所介绍的内容和相关的概念术语，并且完成本讲所布置的示例代码阅读及运行结果分析。

下一讲，介绍对象的构造与状态初始化问题.....